UD00 part 1

4. ACTIVITAT 01 1
5. ACTIVITAT 02 3
6. ACTIVITAT 03 4
7. ACTIVITAT 04 7
8. ACTIVITAT 05 8
9. ACTIVITAT 06 10
10. ACTIVITAT 07 11

4. ACTIVITAT 01

Prepara amb un exemple on expliques com fer en Python 3:
Clonar una llista.

-Shallow copy

```
lista2 = copy.copy(lista1)
```

-Deep copy

```
lista2 = copy.deepcopy(lista1)
```

-Clonación con list( )

```
lista2 = list(lista1)
```

-Slicing (inicio:fin:salto) de distintas formas

```
lista2 = lista1[::] #lo mismo que ::1
lista3 = lista1[0:2] #de la posicion 0 a la 2
lista4 = lista1[0:5:2] #de la posicion 0 a la 5 de dos en dos
```

-Slicing invertido

```
lista2 = lista1[::-1]
```

-Copia de lista (no es una clonación como tal)

```
lista2 = lista1
```

○ **¿Quina és la diferència en Python entre “shallow copy” i “deep copy”?**
Con shallow copy hacemos una copia superficial (solo copiamos un nivel) y se toma como referencia / se comparten los objetos que contiene la lista original, por lo que al modificar la lista original se modifica también la lista clonada, en cambio con deep copy se hace también una copia de los objetos internos, haciendo que la copia sea completamente independiente, por lo que no se aplican los cambios y la lista clonada sigue tal cual clonamos la original.

Con listas de números simples no veríamos una diferencia clara, para comprobarlo vamos a utilizar listas de listas / matrices:

-En este ejemplo con shallow copy podremos comprobar que cuando modificamos una posición de la lista 1 se modificará también de la lista 2:

```
lista1 = [[10, 20], [20, 30]]
lista2 = copy.copy(lista1)
lista1[0][0] = 1
print('Modificando lista1[0][0] = 1 para ver que sucede con lista2: ')
print('Con "Shallow Copy" => lista 1: '+str(lista1)+', lista 2:
'+str(lista2))
```

```
ignguiltar@ORJ1:~/Escritorio/Python$ /usr/bin/python3 /home/ignguiltar/Escritorio/Python/actEv1.py
Modificando lista1[0][0] = 1 para ver que sucede con lista2:
Con "Shallow Copy" => lista 1: [[1, 20], [20, 30]], lista 2: [[1, 20], [20, 30]]
```

-En el mismo ejemplo pero con deep copy podemos comprobar cómo solo se modifica la lista 1:

```
lista1 = [[10, 20], [20, 30]]
lista2 = copy.deepcopy(lista1)
lista1[0][0] = 1
print('Modificando lista1[0][0] = 1 para ver que sucede con lista2: ')
print('Con "Deep Copy" => lista 1: '+str(lista1)+', lista 2:
'+str(lista2))
```

```
ignguiltar@ORJ1:~/Escritorio/Python$ /usr/bin/python3 /home/ignguiltar/Escritorio/Python/actEv1.py
Modificando lista1[0][0] = 1 para ver que sucede con lista2:
Con "Deep Copy" => lista 1: [[1, 20], [20, 30]], lista 2: [[10, 20], [20, 30]]
```

● Afegir un element a una llista.

```
lista = [5, 2, 3, 4, 1]
print("Lista original: ",lista)
lista.append(8)
print("Lista con el 8 añadido: ",lista)
```

```
ignguiltar@ORJ1:~/Escritorio/Python$ /usr/bin/python3 /home/ignguiltar/Escritorio/Python/actEv1.py
Lista original:  [5, 2, 3, 4, 1]
Lista con el 8 añadido:  [5, 2, 3, 4, 1, 8]
```

```
colores = ['rojo', 'verde', 'azul', 'rojo', 'azul']
colores.append('nacho')
```

```
ignguiltar@ORJ1:~/Escritorio/Python$ /usr/bin/python3 /home/ignguiltar/Escritorio/Python/actEv1.py
['rojo', 'verde', 'azul', 'rojo', 'azul', 'nacho']
```

● Llevar un element a una llista.

```
colores = ['rojo', 'verde', 'azul', 'rojo', 'azul']
```

```
colores.pop(4)
```

```
ignglitar@ORJ1:~/Escritorio/Python$ /usr/bin/python3 /home/ignglitar/Escritorio/Python/actEv1.py
['rojo', 'verde', 'azul', 'rojo']
```

```
colores = ['rojo', 'verde', 'azul', 'rojo', 'azul']
colores.remove("verde")
```

```
ignglitar@ORJ1:~/Escritorio/Python$ /usr/bin/python3 /home/ignglitar/Escritorio/Python/actEv1.py
['rojo', 'azul', 'rojo', 'azul']
```

● Crear una nova llista amb els 4 últims elements d'una llista.

```
frutas = ["manzana", "plátano", "naranja", "pera", "kiwi", "fresa"]
ultimas_frutas = frutas[-4:]
print(ultimas_frutas)
```

```
ignglitar@ORJ1:~/Escritorio/Python$ /usr/bin/python3 /home/ignglitar/Escritorio/Python/actEv1.py
['taronja', 'pera', 'kiwi', 'maduixa']
```

● Convertir les paraules d'una cadena (separades per espai) a una llista.

```
cadena = "Nacho Gil Tarrega"
palabras = cadena.split()
print(palabras)
```

```
ignglitar@ORJ1:~/Escritorio/Python$ /usr/bin/python3 /home/ignglitar/Escritorio/Python/actEv1.py
['Nacho', 'Gil', 'Tarrega']
```

● Comentaris amb una línia.

```
# Esto es un comentario de una sola línea - Nacho Gil
```

● Comentaris multilíni

```
"""
Este es un comentario
multilínea
Nacho Gil
"""
```

5. ACTIVITAT 02

En Python 3 els tipus simples passen per valor i els compostos per referència.
Crea un exemple amb 3 funcions que:

● Reva 2 números i torne la suma.

```
def suma(a, b):
    suma = a + b
    return (suma)
```

```
a = int(input("Dame el número 1: "))
b = int(input("Dame el número 2: "))
resultado = suma(a, b)
print("La suma es = "+resultado)
```

```
ignguiltar@ORJ1:~/Escritorio/Python$ /usr/bin/python3 /home/ignguiltar/Escritorio/Python/actEv1.py
Dame el número 1: 5
Dame el número 2: 3
La suma es =  8
```

● Reva una llista i modifique eixa mateixa llista (referència) doblant els valors de tots els elements. No ha de retornar res.

```
def doblar_valor_lista(lista):
    for i in range(len(lista)):
        lista[i] = lista[i] * 2

numeros = [1, 2, 3, 4, 5]
doblar_valor_lista(numeros)
print("Lista con valores doblados",numeros)
```

```
ignguiltar@ORJ1:~/Escritorio/Python$ /usr/bin/python3 /home/ignguiltar/Escritorio/Python/actEv1.py
Lista con valores doblados [2, 4, 6, 8, 10]
```

● Reva una llista i torne una còpia de la llista mateixa llista (referència) doblant els valors detots els elements. La llista original no hi ha de modificar-se

```
def doblar_lista_copia(lista):
    copia = lista.copy()
    for i in range(len(copia)):
        copia[i] = copia[i] * 2

    return copia

numeros = [1, 2, 3, 4, 5]
listaCopia=doblar_lista_copia(numeros)
print("Lista copiada con valores doblados: ",listaCopia)
```

```
ignguiltar@ORJ1:~/Escritorio/Python$ /usr/bin/python3 /home/ignguiltar/Escritorio/Python/actEv1.py
Lista copiada con valores doblados:  [2, 4, 6, 8, 10]
```

6. ACTIVITAT 03

Partint d'un context en què volem emmagatzemar un usuari i la seua contrasenya. Fes un exemple
que explica com es faria:
● Utilitzant una llista.

● Utilitzant un diccionari.
En omplir-se, les contrasenyes han de passar-se a un format Hash (per exemple SHA https://recursospython.com/guias-y-manuales/hashlib-md5-sha/). L'exemple ha d'omplir la llista amb 5 usuaris/contrasenya i fer dues consultes

Utilizando una lista:

```python
import hashlib

def pruebaHash(clave):
    return hashlib.sha256(clave.encode()).hexdigest()


# Lista con 5 usuarios y contraseñas
listaUsuarios = [
    ["Nacho", pruebaHash("123contraseña")],
    ["Oscar", pruebaHash("123profesor")],
    ["Jorge", pruebaHash("dam2026")],
    ["Sergio", pruebaHash("sergio1990")],
    ["Uje", pruebaHash("python3")]
]

print("Lista de usuarios y contraseñas:")
print(listaUsuarios)


# Función
def comprobar_usuario(nombre, clave):
    hashClave = pruebaHash(clave)

    for usuarioActual in listaUsuarios:
        if usuarioActual[0] == nombre and usuarioActual[1] == hashClave:
            return True

    return False


# Dos consultas
print("Consulta 1:")
print(comprobar_usuario("Nacho", "123contraseña"))

print("Consulta 2:")
print(comprobar_usuario("Nacho", "claveIncorrecta"))
```

Aquí podemos comprobar como las contraseñas están en formato Hash, además de cómo nos devuelve True en la primera consulta si introducimos bien la contraseña y cómo nos devuelve False en la segunda consulta si la introducimos de forma incorrecta

```
ignguiltar@ORJ1:~/Escritorio/Python$ /usr/bin/python3 /home/ignguiltar/Escritorio/Python/actHash.py
Lista de usuarios y contraseñas:
[['Nacho', '05ed97df0ffa8ee6cf9cb7ec0f785c1f32e5163da3ef3a75faf22a8745bf0904'], ['Oscar', '4dd803c
3b8e4c820210f751800c64742cb4fe7cb7fc7f0425a107a932'], ['Jorge', 'f8964b7411cbcd3c9e1cad3233c2ee6ce
74de5ad7f701aff8a8000f69'], ['Sergio', 'e94acde5d0e2f8412ae4e5ad0e206b47cfcb3c71f4b1819e0f2ca9457
], ['Uje', 'c1cc69e61c0f1c7ade8df0f2994e582e7c1f2c57d1ec192a0baf9f96b7739d9d']]
Consulta 1:
True
Consulta 2:
False
```

Utilizando un diccionario:

```
# En este caso el usuario será la clave y el Hash de la contraseña será
el valor

diccionarioUsuarios = {
    "Nacho": pruebaHash("123contraseña"),
    "Oscar": pruebaHash("123profesor"),
    "Jorge": pruebaHash("dam2026"),
    "Sergio": pruebaHash("sergio1990"),
    "Uje": pruebaHash("python3")
}

print("Diccionario de usuarios y contraseñas:")
print(diccionarioUsuarios)


# Función
def verificar_diccionario(nombre, clave):
    hashClave = pruebaHash(clave)

    if nombre in diccionarioUsuarios:
        if diccionarioUsuarios[nombre] == hashClave:
            return True

    return False


# Hacemos dos consultas
print("Consulta 1:")
print(verificar_diccionario("Oscar", "123profesor"))

print("Consulta 2:")
print(verificar_diccionario("Oscar", "claveIncorrecta"))
```

Aquí podemos comprobar como las contraseñas están en formato Hash, además de cómo nos devuelve True en la primera consulta si introducimos bien la contraseña y cómo nos devuelve False en la segunda consulta si la introducimos de forma incorrecta

```
ignguiltar@ORJ1:~/Escritorio/Python$ /usr/bin/python3 /home/ignguiltar/Escritorio/Python/actHash.py
Diccionario de usuarios y contraseñas:
{'Nacho': '05ed97df0ffa8ee6cf9cb7ec0f785c1f32e5163da3ef3a75faf22a8745bf0904', 'Oscar': '4dd803c89ba
e4c820210f751800c64742cb4fe7cb7fc7f0425a107a932', 'Jorge': 'f8964b7411cbcd3c9e1cad3233c2ee6ceb3204f
ad7f701aff8a8000f69', 'Sergio': 'e94acde5d0e2f8412ae4e5ad0e206b47cfcb3c71f4b1819e0f2ca94579334a16',
: 'c1cc69e61c0f1c7ade8df0f2994e582e7c1f2c57d1ec192a0baf9f96b7739d9d'}
Consulta 1:
True
Consulta 2:
False
```

7. ACTIVITAT 04

Explica amb exemples com funcionen els operadors “is”, “not”, “in” en Python 3.

-El operador is se utiliza para comprobar si dos variables hacen referencia al mismo objeto

```
lista1 = [1, 2, 3]
lista2 = lista1
print(lista1 is lista2)
```

Esto ocurre porque ambas variables hacen referencia a la misma lista:

```
ignguiltar@ORJ1:~/Escritorio/Python$ /usr/bin/python3 /home/ignguiltar/Escritorio/Python/actEv1.2.py
True
```

En cambio aquí las dos listas tienen los mismos valores, pero son objetos diferentes.

```
lista1 = [1, 2, 3]
lista2 = lista1.copy()
print(lista1 is lista2)
```

```
ignguiltar@ORJ1:~/Escritorio/Python$ /usr/bin/python3 /home/ignguiltar/Escritorio/Python/actEv1.2.py
False
```

-El operador not sirve para negar una condición. Es decir, convierte True en False y False en True

```
edad = 20
print(not edad < 18)
```

La condición edad < 18 es False, pero al utilizar not, se convierte en True.

```
ignguiltar@ORJ1:~/Escritorio/Python$ /usr/bin/python3 /home/ignguiltar/Escritorio/Python/actEv1.2.py
True
```

También podemos utilizarlo para comprobar que un elemento no está en una lista:

```
frutas = ["manzana", "pera", "plátano"]
```

```
print("naranja" not in frutas)
```

Como "naranja" no está en la lista, la condición es verdadera.

```
ignglitar@ORJ1:~/Escritorio/Python$ /usr/bin/python3 /home/ignglitar/Escritorio/Python/actEv1.2.py
True
```

-El operador in sirve para comprobar si un elemento pertenece a una colección, como una lista, una cadena o un diccionario.

```
frutas = ["manzana", "pera", "plátano"]
print("pera" in frutas)
```

```
ignglitar@ORJ1:~/Escritorio/Python$ /usr/bin/python3 /home/ignglitar/Escritorio/Python/actEv1.2.py
True
```

También podemos utilizar in con cadenas:

```
texto = "Nacho Gil"
print("Nacho" in texto)
```

```
ignglitar@ORJ1:~/Escritorio/Python$ /usr/bin/python3 /home/ignglitar/Escritorio/Python/actEv1.2.py
True
```

8. ACTIVITAT 05

Posa un exemple de com passar diversos paràmetres desde consola a un programa Python 3.

Con sys.argv podemos recoger los parámetros desde la consola cuando ejecutamos el programa, el parámetro 0 siempre será el nombre del programa

```
programa = sys.argv[0]
nombre = sys.argv[1]
edad = sys.argv[2]
ciudad = sys.argv[3]

print("Nombre del programa:", nombre)
print("Nombre:", nombre)
print("Edad:", edad)
print("Ciudad:", ciudad)
```

Para pasar los parámetros escribiremos en el terminal python3 + el nombre del programa + los parámetros que queramos introducir

```
ignglitar@ORJ1:~/Escritorio/Python$ python3 act1.5.py Nacho 20 Valencia
Nombre del programa: Nacho
Nombre: Nacho
Edad: 20
Ciudad: Valencia
```

Posa un exemple de com fer "sobrecàrrega de funcions" (funcions que poden rebre diverso números de paràmetres), incloent-hi el cas de què el nombre de paràmetres no siga definit.

El * permite que la función reciba cualquier número de parámetros:

```python
def sobrecarga(*parametros):
    print("Parámetros recibidos:")

    for parametro in parametros:
        print(parametro)

sobrecarga(1)
sobrecarga(1,2)
sobrecarga(1,2,3)
```

```
ignglltar@ORJ1:~/Escritorio/Python$ /usr/bin/python3 /home/ignglltar/Escritorio/Python/act1.5.py
Parámetros recibidos:
1
Parámetros recibidos:
1
2
Parámetros recibidos:
1
2
3
```

```python
def sobrecarga(*parametros):
    print("Parámetros recibidos:")

    for parametro in parametros:
        print(parametro)

sobrecarga("Nacho")
sobrecarga("Nacho","Gil")
sobrecarga("Nacho","Gil","Tarrega")
```

```
ignglltar@ORJ1:~/Escritorio/Python$ /usr/bin/python3 /home/ignglltar/Escritorio/Python/act1.5.py
Parámetros recibidos:
Nacho
Parámetros recibidos:
Nacho
Gil
Parámetros recibidos:
Nacho
Gil
Tarrega
```

En este caso podemos ver que suma todos los parámetros que se van añadiendo aunque el número de estos no están definidos

```python
def sumar(*numeros):
    resultado = 0

    for numero in numeros:
        resultado += numero

    return resultado

print(sumar(2,3))
print(sumar(2,3,5))
print(sumar(2,3,5,8))
```

```
ignglitar@ORJ1:~/Escritorio/Python$ /usr/bin/python3 /home/ignglitar/Escritorio/Python/act1.5.py
5
10
18
```

9. ACTIVITAT 06

Crea un llistat en el qual cada element d'eixa llista siga una llista amb dos valors: mida i pes.
Utilitzant https://docs.python.org/3/howto/sorting.html i las "key functions", fer que aquesta llista s'ordene per major altura i en cas d'igualtat, per menor pes.
Explica en comentaris que és realment la "key function". Pista: en l'ajuda diuen "The value of the key parameter should be a function (or other callable) that takes a single argument and returns a key to use for sorting purposes. This technique is fast because the key function is called exactly once for each input record."

```python
personas = [
    [180, 75],
    [165, 60],
    [180, 70],
    [175, 80],
    [165, 55]
]

print("Lista original:")
print(personas)


# La key function es una función que recibe cada elemento
# de la lista y devuelve una clave que Python utiliza
# para decidir el orden de los elementos.
#
```

```python
# En este caso, primero se utiliza la altura (persona[0])
# y después el peso (persona[1]).
#
# La altura lleva un signo - para ordenarla de mayor a menor.
# El peso no lleva signo -, por lo que se ordena de menor a mayor
# cuando dos personas tienen la misma altura.

def ordenar_persona(persona):
    return (-persona[0], persona[1])


# sorted() crea una nueva lista ordenada utilizando
# la función ordenar_persona como key function.

personasOrdenadas = sorted(personas, key=ordenar_persona)

print("Lista ordenada:")
print(personasOrdenadas)

print(personasOrdenadas)
```

```
igngiltar@ORJ1:~/Escritorio/Python$ /usr/bin/python3 /home/igngiltar/Escritorio/Python/act1.6.py
Lista original:
[[180, 75], [165, 60], [180, 70], [175, 80], [165, 55]]
Lista ordenada:
[[180, 70], [180, 75], [175, 80], [165, 55], [165, 60]]
```

10. ACTIVITAT 07

Defineix la classe Car en Python 3. La classe tindrà com atributs "matrícula" (numèrica) i "color".
Crea un mètode imprimir, i a més dos mètodes que vulgues.
En segon lloc, fes que el programa demane un número "n" per teclat i es creen "n" instàncies de la classe, on cada instància:
● Cada "matrícula" tindrà un número consecutiu des d'1 fins a "n".
● El "color" serà per a cada instància un color aleatori obtingut d'aquest llistat ["red", "white", "black", "pink", "blue"]
Finalment, el programa haurà d'imprimir els valors de les 10 primeres instàncies. En cas que "n" siga menor que 10, només imprimirà "n" instàncies.

```python
import random

class Car:

    def __init__(self, matricula, color):
        self.matricula = matricula
```

```
        self.color = color

    def imprimir(self):
        print("Matrícula:", self.matricula)
        print("Color:", self.color)

    def arrancar(self):
        print("El coche con matrícula", self.matricula, "ha arrancado.")


    def parar(self):
        print("El coche con matrícula", self.matricula, "se ha parado.")


# Lista de colores
colores = ["red", "white", "black", "pink", "blue"]


n = int(input("¿Cuántos coches quieres crear? "))


coches = []


for i in range(1, n + 1):

    color = random.choice(colores)

    coche = Car(i, color)

    coches.append(coche)


cantidad = min(n, 10)

for i in range(cantidad):

    print("Coche", i + 1)

    coches[i].imprimir()

    print()
```

```
ignguiltar@ORJ1:~/Escritorio/Python$ /usr/bin/python3 /home/ignguiltar/Escritorio/Python/act1.7.py
¿Cuántos coches quieres crear? 3
Coche 1
Matrícula: 1
Color: white

Coche 2
Matrícula: 2
Color: pink

Coche 3
Matrícula: 3
Color: blue
```